A Real-Time Traffic Digital Twin from Public CCTV: Vision-Based Detection, Map-Anchored Localization, and Self-Calibrating Speed Estimation

Minsoo Ahn
Department of Computer Science
Stony Brook University
minsoo.ahn@stonybrook.edu

Dahyun Kwon
Department of Computer Science
Stony Brook University
dahyun.kwon@stonybrook.edu

***Abstract*—We present a real-time traffic digital twin that converts live public CCTV feeds into an interactive, map-anchored view of road traffic. From a single clicked camera the system streams HLS video, detects vehicles with YOLO26 [1]—an NMS-free end-to-end detector (YOLOv8 [2] retained as legacy fallback)—and tracks them with a pluggable multi-object tracker (BoxMOT), and converts each vehicle's pixel position to a geographic coordinate through a perspective homography. Because public CCTV cameras are uncalibrated and their intrinsics are unknown, accurate ground-plane mapping and speed estimation are the central challenges. We address them with three complementary mechanisms: (i) a two-stage *curved* transform that maps pixels along the true road centreline obtained from the national node-link road network rather than a flat plane; (ii) a five-stage speed estimator combining a physics-based outlier guard, least-squares regression over a sliding window, and per-track exponential smoothing; and (iii) an online self-calibration loop that compares the measured speed against the official ITS segment speed and learns a per-camera correction factor. The system additionally provides multi-camera background monitoring, camera-level congestion clustering, and a SQLite time-series history with charting. We describe the architecture and the key algorithms, and report latency, throughput, tracking-stability, and speed metrics produced by a measurement harness over live streams.**

***Index Terms*—digital twin, intelligent transportation systems, object detection, multi-object tracking, homography, perspective transform, speed estimation, real-time visualization.**

I. Introduction

Cities expose thousands of public traffic cameras, but the raw feeds are difficult to interpret at scale: an operator watching a video grid cannot read off vehicle counts, speeds, or where congestion is forming on a map. A *digital twin*—a live, geo-referenced model of the road state—turns those feeds into actionable information. The technical obstacle is that public CCTV cameras are *uncalibrated*: their height, pitch, focal length, and exact heading are unknown, so projecting an image detection onto the ground plane (and therefore measuring distance and speed) is ill-posed without per-camera calibration.

This paper describes a system that ingests Korean ITS CCTV streams and produces a real-time map-anchored twin. Our contributions, from a systems and algorithms standpoint, are:

- **Map-anchored localization.** We snap each camera onto the national node-link road network, extract the road centreline, and map pixels along the *curve* of the road in two stages, rather than assuming a flat homographic plane. This keeps vehicle markers on the road even on bends.
- **Robust speed estimation without ground truth.** A five-stage filter (duplicate skip, physics jump guard, sliding-window least-squares regression, per-track exponential smoothing with spike rejection, and scale correction) produces stable speeds despite detection jitter and tracker ID churn.
- **Online self-calibration.** Because absolute scale is unknowable from an uncalibrated camera, we learn a per-camera `speed_scale` by comparing our measured rolling average against the ITS official segment speed, treating it as a soft reference (α=0.99) to account for ITS measurement uncertainty (5-min aggregation, 1 km granularity, possible direction mismatch).
- **A complete, resilient pipeline.** Automatic lane-based calibration, manual 4-point calibration, dual-path inference (browser canvas and server-side HLS) over a shared detector, multi-camera background monitoring, congestion clustering, and a historical analytics store.

The rest of the paper is organized as follows. Section II reviews related work. Section III presents the system architecture. Section IV details the algorithms. Section V reports the measurements. Section VI discusses limitations, and Section VII concludes.

II. Related Work

Object detection and tracking. Modern traffic vision builds on real-time detectors; we use YOLO26 [1] as the default detector—an NMS-free end-to-end architecture that produces more consistent per-frame bounding-box positions than NMS-based models such as YOLOv8 [2], reducing the ground-contact jitter that propagates into speed estimates. It also simplifies the TensorRT FP16 engine graph (no NMS layer) [3], lowering inference latency and its variance; YOLOv8 is retained as a legacy fallback. For tracking we use the BoxMOT framework [4], which exposes ByteTrack [5], BoT-SORT [6], and OC-SORT [7]; appearance re-identification

uses OSNet [8]. Trackers without appearance ReID suffer ID switches under occlusion, which we mitigate with a lightweight position-based ID stabilizer.

Camera-to-ground mapping. Mapping image points to a ground plane is a homography problem [9], typically solved from four correspondences with OpenCV [10]. For *uncalibrated* cameras, vanishing-point and lane-geometry cues are commonly used to recover pose. Our work combines a 4-point manual mode, a vanishing-point/lane auto-calibration, and—novel in this context—a road-centreline-following transform driven by an external road-network dataset [11].

ITS dashboards and data. National ITS portals [12] publish CCTV streams and aggregated segment speeds, but typically as separate views. We *fuse* the segment speed back into the pipeline as a calibration signal. Congestion grouping uses density-based clustering [13] and convex hulls [14]; service levels follow the Highway Capacity Manual [15]. The visualization uses deck.gl [16] over MapLibre [17], served by FastAPI [18].

III. System Design and Architecture

The backend (FastAPI/Uvicorn) and a React/deck.gl frontend communicate over REST and WebSocket. Two inference paths can produce per-frame analytics and *share a single detector*; a connection counter coordinates them so that the multi-object tracker is never driven by two interleaved frame streams (Fig. 1).

Browser path (`/ws/detect`). The frontend plays the HLS stream through a CORS proxy, captures canvas frames (JPEG, $\leq$640 px, $\approx$33 ms interval, at most two in flight), and sends them to the backend, which runs detection, tracking, localization, and analytics, then returns an annotated frame and broadcasts the analytics JSON to all clients.

Server path (`live_loop`). The backend can also read the HLS stream directly with OpenCV/FFmpeg and run the same pipeline. It is gated twice: it skips tracking while a browser detection client is active (to protect the shared tracker), and it skips all GPU work when no one is watching (the frontend reports tab visibility).

Background services. Periodic loops refresh expiring HLS tokens (30 min), poll the ITS segment speed for self-calibration (5 min), and snapshot all monitored cameras into a SQLite history while recomputing congestion clusters (30 s). To prevent the FOV polygon from flickering during auto-calibration, a slow EMA filter stabilizes near/far distance and road-width estimates, holding the initial accepted value fixed until sufficient samples accumulate. ROI edits trigger an immediate GPS ring recomputation and broadcast to keep the map view synchronized. The resulting map view is shown in Fig. 2.

IV. Implementation

A. Detection and Tracking

The detector loads the fastest available backend—TensorRT FP16 engine, then ONNX Runtime, then PyTorch—and exports `.pt`→`.engine` on first run when a GPU is present. A tracker *tier* is chosen by GPU memory: ByteTrack (CPU/low-VRAM), OC-SORT (occlusion-robust, no ReID), or BoT-SORT/DeepOC-SORT with OSNet appearance ReID on larger GPUs. For the non-ReID tiers we add two robustness layers: `_dedup_tracks` merges duplicate boxes (IoU >0.3 or centre distance <40 px, keeping the older ID), and an `IDStabilizer` re-binds a vehicle's previous ID after a brief miss using nearest-centre matching ($\leq$80 px) with a two-pass scheme and stale-mapping purge so display IDs stay stable and compact. Fig. 3 shows the detection and tracking overlay.

Algorithm 1 Two-stage curved pixel→GPS transform

Require: pixel (u, v); metric homography H_m; centreline *road_pts*; *snap_along*; direction sign s

$(x_m, y_m) \leftarrow H_m \cdot (u, v)$
$(dx, dy) \leftarrow (x_m, y_m) - \mathit{snap}_m$ ▷ ENU from snap
$d_\parallel \leftarrow dx \sin b + dy \cos b$; $d_\perp \leftarrow dx \cos b - dy \sin b$
$\mathit{arc} \leftarrow \mathit{snap\_along} + s \cdot \max(0, d_\parallel)$
$(\mathit{lat}, \mathit{lon}) \leftarrow$ InterpCenterline(arc)
$b_\ell \leftarrow$ RoadBearingAt(arc)
apply $d_\perp$ perpendicular to b_ℓ **return** $(\mathit{lat}, \mathit{lon})$

B. Map-Anchored Localization

On camera switch we query the national node-link network: we snap the camera position perpendicularly onto the nearest road polyline, extract $\pm$150 m of centreline, and—because each direction is stored as a separate one-way link—we locate the reverse carriageway and average the two snaps (when 2–40 m apart) to recover the true road centre. The chosen road's speed limit, lane count, and width (lanes $\times$ 2 $\times$ lane-width) are attached to the camera; OSM Overpass provides a width refinement when available.

Given the centreline, the *two-stage curved transform* maps a pixel (u, v) to GPS along the road (Algorithm 1): stage one maps the pixel to local metres via the metric homography; stage two decomposes the displacement from the snap point into along-road and lateral components, advances that arc length along the centreline, interpolates the on-road position, and re-applies the lateral offset perpendicular to the *local* road bearing. This follows real road curvature, unlike a single flat homography.

When no manual calibration exists, an automatic procedure estimates a homography from one frame: Canny edges in the lower image, Hough line filtering, multi-row sampling of left/right road edges, least-squares edge fits, vanishing-point intersection, a pitch/height estimate from a pinhole model, and a trapezoid-to-GPS homography. Direction (which way the camera looks along the road) is resolved by matching the *image* road-curve sign against the *map* curve sign, falling back to a vanishing-point heading test. A manual 4-point mode remains available for maximum accuracy. Fig. 4 shows an automatic calibration result.

The primary auto-calibration path is a road-model *pose solver* (`camera_pose.solve_pose`): the sampled lane edges, vanishing point, and NodeLink road width are fitted to a four-parameter pinhole-camera model (H, pitch, yaw,

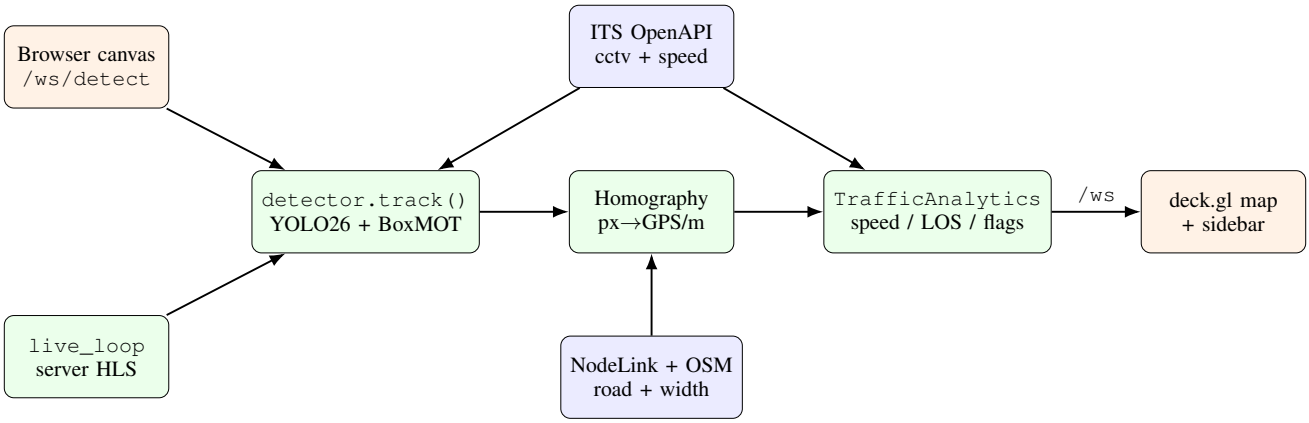

Fig. 1. System architecture. The browser path (`/ws/detect`) and the server path (`live_loop`) share one detector; a counter prevents concurrent tracker calls. External data (ITS CCTV/speed, NodeLink, OSM) feeds localization and self-calibration.

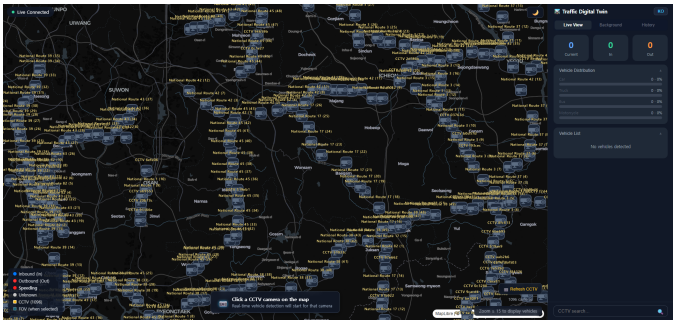

Fig. 2. Map view: public CCTV cameras as status-coloured icons on the vector map; the selected camera shows its field-of-view polygon and the located vehicles.

Fig. 3. Detection and tracking on a live CCTV frame: bounding boxes, vehicle class, and stabilized track IDs.

lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1 loss). A per-row reprojection residual gates the output (<8 px); on success the solved pose is converted to four image–GPS corner pairs and used to build both homographies. The pose is then persisted per camera to `camera_pose.json` so each session refines the prior, with a cold-start fallback chain

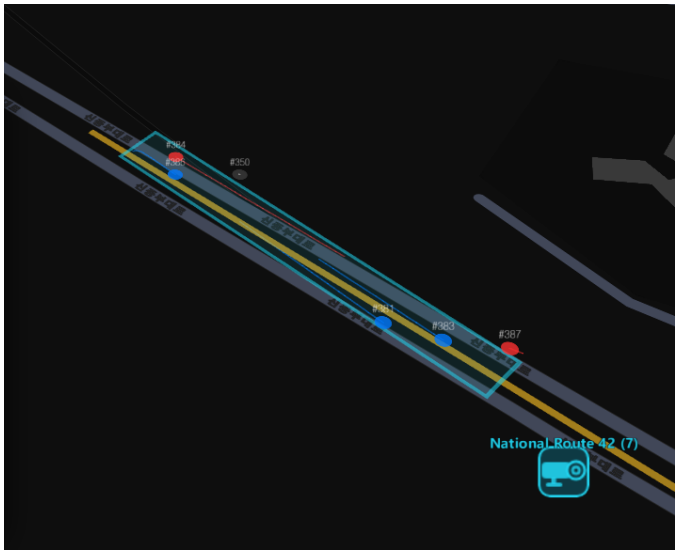

Fig. 4. Automatic calibration: detected lane edges and the estimated vanishing point yield the ground homography and the field-of-view footprint.

(saved prior → vehicle-bbox rough pose → GPS-grid). The trapezoid heuristic above serves only as the fallback when the residual exceeds the quality gate or lane detection fails.

After GPS coordinates are assigned, a two-step post-processing pass reduces visual noise. Per-track EMA smoothing eliminates lateral jitter while preserving along-road motion; a jump threshold resets the filter after occlusion gaps to handle track re-appearances correctly. A lane-offset pass then laterally separates inbound and outbound vehicles perpendicular to the local road bearing, placing each direction on its correct side of the road centreline for Korean right-hand traffic.

C. Speed Estimation

Speed is distance over time, and both terms are error-prone: homography distance is noisy at the far field, and HLS buffering makes naive frame timing wrong. We use a monotonic clock derived from stream presentation timestamps (falling back to

Algorithm 2 Per-track speed estimation (`analytics._speed`)

Require: window W, prior EMA v_e, $\sigma \leftarrow$ `speed_scale`

1: **if** duplicate position **then** skip ▷ dedup
2: **if** *step* $> (V_{\max}/3.6\sigma)\, dt \cdot 1.5$ **then** drop sample, keep W ▷ jump guard
3: **if** $dt > 2\,\text{s}$ **then** clear W ▷ track gap
4: fit $\hat{v}$ by OLS on W (0.7 s); **if** $\text{disp}(W) < 0.5\,\text{m}$ **then** $\hat{v} \leftarrow 0$ ▷ OLS
5: $v_e \leftarrow (1-\alpha)v_e + \alpha\hat{v}\sigma$, α=0.35; reject spike; decay on stop ▷ EMA
6: **output** v_e; *is_speeding* $\leftarrow v_e > 1.10 \cdot$ *limit* ▷ output

TABLE I
KEY SPEED/ANALYTICS CONSTANTS (`CONFIG.PY`).

Parameter	Value
Speed window	0.7 s (W=round(0.7s×FPS))
EMA α	0.35
Jitter threshold	0.5 m
Min speed (else 0)	5 km/h
Max plausible $V_{\max}$	180 km/h
GC grace	30 frames
Bottleneck dwell	150 frames (≈5 s)
Parked dwell	300 frames (≈10 s)
Speeding threshold	$v_e > 1.10 \times$ limit
LOS thresholds A–E	≤3,6,9,12,15 (else F)
Background busy	>6 vehicles
Background congested	>14 vehicles
Cluster ε	500 m (haversine)

wall-clock), and the five-stage estimator of Algorithm 2 with the constants in Table I.

D. Online Self-Calibration

The absolute distance scale of an uncalibrated camera is unknowable, so a residual scale error persists after geometric calibration. Every five minutes we fetch the ITS official segment speed near the camera and update a per-camera correction factor `speed_scale`. The ITS signal carries inherent uncertainty: measurements are aggregated over approximately 1 km road segments at 5-minute intervals, the reported direction may not correspond to the camera's field of view, and the figure reflects a broad spatial average rather than the camera's precise observation zone. Accordingly, the calibration loop treats the ITS speed as a *soft reference*:

$$\textit{target} = \textit{scale} \cdot \frac{v_{\text{ITS}}}{\bar{v}_{\text{ours}}}, \quad \textit{scale} \leftarrow (1-\alpha)\,\textit{scale} + \alpha\,\textit{target}, \quad (1)$$

with α=0.99 (very slow adaptation), clamped to $[0.3, 5.0]$, using a 10-minute rolling average (≥50 samples) and a coefficient-of-variation guard (≤0.4) to skip volatile traffic. The factor is persisted per camera and accumulates across sessions without resetting the geometric calibration.

E. Aggregation, Congestion, and History

Direction is primarily determined by projecting each vehicle's inter-frame displacement onto the road bearing axis with an EMA deadzone filter; a horizontal LineZone crossing serves as a fallback when road bearing is unavailable, and also provides In/Out counts. Dwell counting flags bottlenecks and parked vehicles (the latter remembered by pixel position so they survive ID changes), and a Level-of-Service grade is assigned from the active vehicle count. Background monitoring polls additional cameras every 8 s with detection only; cameras with more than 6 vehicles are marked *busy* and those with more than 14 are marked *congested*; such cameras are then clustered with a haversine density-based scan (ε=500 m) and outlined with a convex hull. A WAL-mode SQLite store records 30 s snapshots (14-day retention) and serves bucketed time series, peak-time detection, and CSV export. Fig. 5 shows the congestion overlay.

Fig. 5. Camera-level congestion: monitored cameras are recoloured (normal/busy/congested) and adjacent congested cameras are grouped into a severity-shaded region.

F. Visualization

The deck.gl map layers congestion polygons, vehicle trails, the FOV polygon, status-coloured camera icons, and vehicle markers. The FOV polygon uses one of three strategies in priority order: the manually clicked GPS ring; a curved corridor that follows the road centreline (mirroring the backend transform); or, when uncalibrated, a default trapezoid.

The sidebar's vehicle list provides *direction-filter tabs* (All / Inbound / Outbound) that filter the displayed rows by the LineZone-derived direction; the per-direction vehicle count is shown in each tab badge, and the min/avg/max speed summary is recomputed from the filtered set. Two collapsible cards (Auto Calibration Estimate, ITS Speed Comparison) include an info button that opens a modal explanation of each metric, adapting to the selected map theme.

TABLE II
PER-STAGE LATENCY (MS) AND THROUGHPUT. FILL FROM EVAL_LATENCY.CSV.

Stage	Mean	Median	p95
YOLO (isolated)	fused into Track stage (TensorRT)		
Track (YOLO+BotSort)	31.1	27.5	46.8
Transform	0.09	0.08	0.20
Analytics	0.18	0.16	0.38
End-to-end	**31.3**	**27.7**	**47.2**
Throughput: 31.9 FPS			

TABLE III
TRACKING STABILITY AND SPEED DISTRIBUTION. FILL FROM EVAL_TRACKING.CSV / EVAL_SPEED.CSV.

Metric	Value
Frames processed	4,559
Unique tracks	328
ID appearances (switch proxy)	683 (2.08×)
Mean track lifetime (frames)	73.9
Median measured speed (km/h)	75.1
% moving observations	70.6 %

V. EVALUATION

Measurements are produced from the real pipeline in two ways. While the system runs (`make dev`), an in-process collector records per-frame stage latency, tracking, speed, and detection statistics and flushes `eval_*.csv` and `eval_summary.json` every 30 s (also on demand via `GET /eval/report`); an offline harness (`backend/evaluate.py`) reproduces the same metrics over a fixed clip for controlled benchmarks. Numbers below are produced on the target hardware.

Setup: model `YOLO26m`, backend `TensorRT`, tracker `BotSort`, source: Goyang Jugyo Underpass, National Rd. 39 (live HLS), 4,559 frames (935.9 s).

Speed accuracy. Because no per-vehicle ground truth exists for public CCTV, we assess speed against the ITS official segment speed for the same camera and time window. Table IV compares the measured 10-minute average against ITS and reports the learned scale factor and convergence.

VI. DISCUSSION AND LIMITATIONS

The homography is accurate inside the calibration quadrilateral and extrapolates with growing error toward the far field; we mitigate this with the dual-matrix calibration, the regression window, lane-based auto-calibration, and the ITS `speed_scale`, but a residual error remains on poorly observed cameras. Automatic calibration assumes a focal length of $1.2\times$ image height (unknown intrinsics), giving a $\pm$15–20 % scale uncertainty per FoV mismatch, and fails when lane markings are not visible (night, rain, dense traffic), falling back to an approximate grid. Road width is inferred from lane count and assumed bidirectional, so one-way roads are over-wide. The nearest node-link road is occasionally the wrong one (e.g., a camera near both a national route and an expressway), which displaces the FOV polygon and vehicle GPS; the CCTV-name road-hint reduces but does not eliminate this. Finally, congestion is clustered at the camera level because background cameras expose only counts, not per-vehicle speeds.

TABLE IV
MEASURED VS. ITS SEGMENT SPEED (PER CAMERA). FILL FROM THE LIVE OUR_AVG_KPH/ITS_SPEED_KPH/SPEED_ERROR_PCT BROADCAST AND SPEED_SCALE.JSON.

Camera	Ours avg (km/h)	ITS (km/h)	scale
(37 cameras accumulated; representative sample)			
Jugyo Underpass (eval)	84.0	—	0.846
Typical range (37 cams)	—	—	0.34 – 2.25

Because the system derives all geographic positions from map geometry rather than on-vehicle GPS receivers, localization accuracy is bounded by the quality of the NodeLink road database and the precision of the perspective projection; errors in map alignment or road-shape approximation propagate directly into vehicle GPS coordinates. Additionally, a camera's effective field of view is strongly shaped by its mounting angle: a steeply pitched installation sees a narrow, elongated strip of road, producing a thin FOV polygon on the map and limiting the usable observation window even after geometric calibration.

A structural limitation is the system's dependence on national ITS infrastructure: camera streams and segment speeds are fetched from a government-operated open API [12] whose availability and authentication policy are outside the operator's control and may be restricted or discontinued. Camera placement and viewing angle are fixed by road authorities and cannot be adjusted; some installations are positioned too far or too close to capture a useful road segment, and extreme viewpoints cannot be recovered through calibration. Adverse conditions—low illumination, rain, lens contamination, and hardware faults—can suppress the visual signal entirely, leaving the pipeline with no basis for detection or calibration. These constraints are inherent to re-using existing infrastructure without modification.

VII. CONCLUSION

We built a real-time traffic digital twin that localizes vehicles on a map from uncalibrated public CCTV and estimates their speed without ground truth. The key ideas are a road-centreline-following two-stage transform, a robust five-stage speed estimator, and an online self-calibration loop driven by official ITS segment speeds, wrapped in a resilient dual-path pipeline with congestion analytics and a historical store. The approach makes raw municipal camera feeds directly usable for situational awareness, and its self-calibration removes the main barrier—per-camera manual setup—to deploying across many cameras.

A distinguishing property is that the system requires *no GPS instrumentation on vehicles*: conventional traffic monitoring relies on embedded GPS loggers or dedicated roadside sensors, both of which demand significant infrastructure investment. By re-purposing cameras that road authorities already operate, the

system tracks multiple vehicle classes simultaneously across multiple views with no additional hardware cost. As public CCTV coverage continues to expand and object detectors advance, this class of vision-only traffic twin becomes increasingly practical at scale. With wider network integration, the same localization principle—positioning objects through perspective projection rather than satellite signal—could extend to GPS-denied environments such as tunnels, underground parking, and indoor logistics facilities; the per-camera track-ID scheme further enables cross-camera vehicle re-identification for a range of fleet management and security applications.

VIII. Contribution

This section states, per author, what was already available, what each of us specifically added, why it is technically different, and the evidence in the codebase. The focus is technical novelty rather than effort percentages. The two authors co-designed the architecture and integrated their parts; the split below reflects primary ownership.

A. Minsoo Ahn — Detection/Tracking, Analytics, and System Infrastructure

What was already there. The starting point was a basic detection pipeline: YOLOv8 [2] identified vehicles frame by frame and a Level-of-Service grade described the road state. Speed was approximated from a simple distance-over-time ratio. This provided frame-level counts and a service grade, but no stable vehicle identities, no reliable per-vehicle speed, and no actionable state classification.

What I added.

- *BoxMOT integration and tracker-tier architecture* (`detector.py`, `model_setup.py`): added multi-object tracking [4] and designed a VRAM-based tier system that automatically selects ByteTrack [5], OC-SORT [7], BoT-SORT [6], or DeepOC-SORT by available GPU memory, so the pipeline scales from a CPU-only machine to a high-end workstation without code changes, balancing FPS and accuracy at each tier.
- *TensorRT inference backend* (`detector.py`): added TensorRT [3] FP16 engine export and an auto-selection chain (engine → ONNX Runtime → PyTorch), raising throughput and reducing latency variance.
- *YOLO26 upgrade* (`detector.py`, `config.py`): migrated the default detector from YOLOv8 to YOLO26 [1], an NMS-free end-to-end architecture, combined with TensorRT to form the current production pipeline. Raised `YOLO_CONF` from 0.25 to 0.30 since post-NMS filtering is no longer available at the detection level.
- *CCTV-based speed estimator* (`analytics.py:_speed`, `main.py:_speed_timestamp_ms`): replaced the naive distance/time formula with the five-stage filter of Algorithm 2 (physics jump guard, sliding-window OLS regression, EMA smoothing with spike rejection, PTS-derived time axis) to address the specific challenges of CCTV-based measurement.
- *Dwell counting — bottleneck and parked-vehicle detection* (`analytics.py`): dwell-frame counting and pixel-position memory to distinguish stopped-in-traffic from genuinely parked vehicles, surviving tracker ID cycling.
- *ID-based tracking accuracy* (`detector.py`): `IDStabilizer` (two-pass remap with stale-mapping purge), `_dedup_tracks`, and empty-detection grace period to prevent ID churn from corrupting per-vehicle state.
- *Backend infrastructure* (`main.py`, `congestion.py`, `history.py`): FastAPI dual-path server with shared-detector coordination, congestion clustering, SQLite history store, and background multi-camera monitoring.
- *Initial calibration concept*: proposed the original 4-point homography and ROI-based filtering idea, which Dahyun then developed into the full automatic calibration pipeline.
- *UI, i18n, and evaluation harness* (frontend, `evaluate.py`, `metrics.py`): analytics panels, vehicle direction-filter table with speed summary, history charts, WebSocket layer, `colorMap.js` colour system, i18n internationalisation, the YOLO detection tab in `CctvPlayer.jsx`, and the offline measurement harness.
- *Movement-vector direction classification* (`analytics.py`): replaced the LineZone-only direction scheme with a continuous movement-based classifier that projects each vehicle's inter-frame displacement onto the road bearing axis and applies an EMA deadzone filter; LineZone crossing is retained as a fallback when road bearing is unavailable.
- *GPS position post-processing pipeline* (`analytics.py`): added per-track EMA position smoothing to remove lateral jitter without distorting along-road motion, and a lane-offset pass that laterally separates inbound and outbound vehicles by the local road bearing perpendicular, producing physically correct lane placement for Korean right-hand traffic.
- *FOV polygon stabilization and ROI broadcast* (`main.py`): introduced a slow EMA filter on auto-calibrated FOV parameters (near/far distance, road width) that locks in the initial accepted estimate and drifts toward new values only after sufficient samples, preventing polygon flicker; ROI edits now trigger an immediate GPS ring recomputation and live broadcast.

Why it is technically different. The goal throughout was not just detection but measurements accurate and versatile enough for real-world use. The GPU-tier design lets the pipeline run from a CPU laptop to a high-end workstation without code changes. The five-stage speed estimator and PTS-based time axis produce stable, quantitative speed from passive CCTV streams—a capability absent from the baseline detection-and-count framework—and the shared-detector coordination lets browser and server inference paths coexist safely. The movement-vector direction classifier removes dependence on LineZone crossing for In/Out labelling, enabling correct

direction assignment from the moment a vehicle enters the frame; combined with the lane-offset and position smoothing passes, the map display reflects physical lane placement rather than noisy projected points.

Evidence. `detector.py` (`IDStabilizer`, `track`, tracker-tier selection); `model_setup.py`; `analytics.py` (`_speed`, dwell/LOS, `_assign_directions`, `_smooth_positions`, `_apply_lane_offset`); `main.py` (`live_loop`, `_speed_timestamp_ms`, `BackgroundMonitor`, `_apply_fov_ema`, `_compute_roi_gps_ring`); `congestion.py`; `history.py`; the detection overlay (Fig. 3), the congestion overlay (Fig. 5), and the measurement harness (`evaluate.py`).

B. Dahyun Kwon — Localization, Calibration, and Map Visualization

What was already there. The original concept was to run YOLO on a Korean ITS camera stream and observe the output. There was no ground-plane mapping, no calibration, no road geometry, and no reliable In/Out vehicle counting—vehicles appeared wherever a flat default homography happened to project them.

What I added.

- *In/Out vehicle counting baseline* (`tracker.py`): built the initial `VehicleTracker` with a horizontal `LineZone` that yields per-frame In/Out counts and per-vehicle direction, providing the foundation for directional traffic analysis.
- *Auto-calibration pipeline (full development)* (`transform.py`, `camera_pose.py`): evolved calibration from a basic trapezoid heuristic to the current road-model pose solver—a four-parameter pinhole fit (H, pitch, yaw, lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1) that persists per camera and refines across sessions. A cold-start fallback chain covers edge cases (saved prior $\rightarrow$ vehicle-bbox pose $\rightarrow$ GPS-grid).
- *ITS speed-based auto-calibration loop* (`analytics.py:calibrate_from_its`): designed and implemented the online self-calibration loop that compares the measured rolling average against the official ITS segment speed and learns a per-camera `speed_scale`, with convergence detection and a coefficient-of-variation guard to skip volatile traffic.
- *NodeLink road network integration* (`nodelink.py`, `osm.py`, `transform.py`): identified the MOCT NodeLink dataset [11] as the geometric anchor and drove its full integration: road snapping, centreline extraction, bidirectional carriageway fix, road-width derivation, and OSM Overpass [19] width refinement.
- *Two-stage curved pixel→GPS transform* (`transform.py:_pixel_to_gps_curved`): arc-length interpolation along the road centreline, keeping vehicle GPS positions correct on curved roads (Algorithm 1).
- *Automatic ROI and single-click camera setup* (`roi_manager.py`, `RoiEditor.jsx`): extended automatic calibration to include ROI estimation, so clicking a camera triggers calibration, ROI, and road-snapping with no manual step.
- *Camera geometry utilization* (`camera_pose.py`): the pose solver explicitly recovers camera height and pitch from lane geometry, making the system self-aware of its viewing geometry without any pre-installed metadata.
- *Map visualization and calibration UI* (`MapView.jsx`, `CalibrationMode.jsx`, `CctvPlayer.jsx` calibration overlay): deck.gl FOV polygon strategies (manual ring / curved corridor / trapezoid), calibration overlay with NodeLink node snapping, and the calibration tab in the CCTV player.

Why it is technically different. The system advanced from raw YOLO output to self-calibrating, road-geometry-aware localization. The pose solver makes calibration reproducible and session-improving rather than a one-off manual step; the curved transform keeps GPS positions physically grounded on non-straight roads; the ITS calibration loop corrects the longitudinal scale error that geometric calibration alone cannot resolve; and the single-click workflow means any new ITS camera can be brought online automatically without manual configuration.

Evidence. `tracker.py` (`VehicleTracker`, `LineZone`); `camera_pose.py` (`solve_pose`, `pose_to_corners`); `transform.py` (`_pixel_to_gps_curved`, `auto_calibrate_from_frame`, `update_from_calibration`); `analytics.py:calibrate_from_its`; `nodelink.py`, `osm.py`, `roi_manager.py`; `MapView.jsx` FOV strategies; `CalibrationMode.jsx`, `RoiEditor.jsx`; the automatic calibration result (Fig. 4) and the map view (Fig. 2).

REFERENCES

[1] Ultralytics, "YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection," *arXiv preprint arXiv:2509.25164*, 2025. [Online]. Available: https://arxiv.org/abs/2509.25164

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023, open-source object detection and tracking framework.

[3] NVIDIA, "TensorRT: High-performance deep learning inference optimizer and runtime," https://developer.nvidia.com/tensorrt, 2023.

[4] M. Broström, "BoxMOT: Pluggable SOTA multi-object tracking modules," https://github.com/mikel-brostrom/boxmot, 2023.

[5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conf. on Computer Vision (ECCV)*, 2022, pp. 1–21.

[6] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," *arXiv preprint arXiv:2206.14651*, 2022.

[7] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[8] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, "Omni-scale feature learning for person re-identification," *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2019.

[9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
[10] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.
[11] Korea Transport Database (KTDB) / Ministry of Land, Infrastructure and Transport, "Standard node-link (MOCT) road network dataset," https://www.its.go.kr/nodelink, 2023.
[12] National Transport Information Center (ITS), Republic of Korea, "ITS open API: Real-time CCTV and traffic information," https://openapi.its.go.kr, 2023.
[13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
[14] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.
[15] Transportation Research Board, "Highway capacity manual (HCM)," *National Research Council, Washington, D.C.*, 2016.
[16] vis.gl / Open Visualization, "deck.gl: WebGL2-powered geospatial visualization framework," https://deck.gl, 2023.
[17] MapLibre Contributors, "MapLibre GL JS," https://maplibre.org, 2023.
[18] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com, 2018.
[19] OpenStreetMap Contributors, "OpenStreetMap and the Overpass API," https://www.openstreetmap.org, 2023.